

# PLAYWRIGHT (JS/TS)

## LAST-MINUTE INTERVIEW CHEAT SHEET

Suriyaprakash | AI-First SDET & Mentor | [LinkedIn](#) · [topmate.io/automation\\_made\\_easy](https://topmate.io/automation_made_easy)

# 1. SETUP & INSTALLATION

## ■ Install & Init

```
# Install Playwright
npm init playwright@latest

# Install manually
npm install -D @playwright/test
npx playwright install # install all browsers
npx playwright install chromium # specific browser
npx playwright install --with-deps # with system deps
```

## ■ playwright.config.ts — Key Config

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30_000, // per-test timeout
  retries: 2, // retry on failure
  workers: 4, // parallel workers
  reporter: 'html',
  use: {
    baseURL: 'https://app.com',
    headless: true,
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
    actionTimeout: 10_000,
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'mobile', use: { ...devices['iPhone 14'] } },
  ],
});
```

## 2. TEST STRUCTURE & HOOKS

### ■ Basic Test

```
import { test, expect } from '@playwright/test';

test.describe('Login', () => {

  test.beforeAll(async ({ browser }) => {
    // runs once before all tests
  });

  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
  });

  test('valid login', async ({ page }) => {
    await page.fill('#user', 'admin');
    await page.fill('#pass', 'pass');
    await page.click('#btn');
    await expect(page).toHaveTitle('Dashboard');
  });

  test.afterEach(async ({ page }) => {
    // cleanup after each test
  });

  test.afterAll(async () => {
    // cleanup once after all tests
  });
});
```

### ■ Test Annotations

| Annotation           | Purpose                   |
|----------------------|---------------------------|
| test.only(...)       | Run only this test        |
| test.skip(...)       | Skip this test            |
| test.fixme(...)      | Mark as broken/todo       |
| test.fail(...)       | Expect test to fail       |
| test.slow(...)       | Triple the timeout        |
| test.describe.only   | Run only this suite       |
| test.describe.skip   | Skip this suite           |
| test.describe.serial | Run suite serially        |
| test.step(...)       | Group steps for reporting |
| test.info()          | Access test metadata      |

■ Use `test.step()` to group actions — shows in HTML report and trace viewer.

### 3. LOCATORS

| Locator               | Syntax                                                     | Notes                                         |
|-----------------------|------------------------------------------------------------|-----------------------------------------------|
| Role<br>(recommended) | <code>page.getByRole('button', { name: 'Submit' })</code>  | Accessibility-first, most robust              |
| Text                  | <code>page.getByText('Welcome')</code>                     | Partial match by default                      |
| Label                 | <code>page.getByLabel('Email')</code>                      | For form inputs with labels                   |
| Placeholder           | <code>page.getByPlaceholder('Enter email')</code>          | Input placeholder text                        |
| Alt Text              | <code>page.getByAltText('Logo')</code>                     | For images                                    |
| Title                 | <code>page.getByTitle('Close')</code>                      | title attribute                               |
| Test ID               | <code>page.getByTestId('submit-btn')</code>                | data-testid attribute                         |
| CSS                   | <code>page.locator('.btn-primary')</code>                  | Standard CSS selector                         |
| XPath                 | <code>page.locator('//button[@id="btn"]')</code>           | XPath selector                                |
| nth                   | <code>page.locator('li').nth(2)</code>                     | Zero-based index                              |
| first / last          | <code>page.locator('li').first()</code>                    | <code>.first()</code> or <code>.last()</code> |
| filter                | <code>page.locator('li').filter({ hasText: 'Buy' })</code> | Filter by text/locator                        |

#### ■ Chaining & Scoping Locators

```
// Chain locators
page.locator('.card').locator('button');

// Filter by text
page.locator('.row').filter({ hasText: 'Active' }).locator('button');

// Filter by locator
page.locator('.row').filter({ has: page.locator('.badge' )});

// Nth match
page.locator('input').nth(0);
page.locator('input').first();
page.locator('input').last();

// Combine: AND / OR
page.locator('button').and(page.locator('[data-active]'));
page.locator('button').or(page.locator('a'));
```

## 4. ACTIONS

### ■ Click & Type

```
await page.click('selector');
await page.dblclick('selector');
await page.fill('#input', 'text'); // clears
first
await page.type('#input', 'text'); // types char
by char
await page.clear('#input');
await page.press('#input', 'Enter');
await page.press('body', 'Control+A');

// Via locator
await page.locator('#btn').click();
await page.locator('#input').fill('val');
await page.locator('#input').press('Tab');
```

### ■ Hover, Drag & Drop

```
await page.hover('selector');
await page.locator('#src').dragTo(
page.locator('#tgt'));

// With options
await page.locator('#src').dragTo(
page.locator('#tgt'),
{ sourcePosition: { x:10, y:5 },
targetPosition: { x:20, y:5 } });
```

### ■ Select, Check & Upload

```
// Dropdown
await page.selectOption('#sel', 'value');
await page.selectOption('#sel', { label: 'Option'
});
await page.selectOption('#sel', { index: 2 });
// Multi-select
await page.selectOption('#sel', ['v1','v2']);

// Checkbox / Radio
await page.check('#checkbox');
await page.uncheck('#checkbox');
await page.locator('#radio').check();

// File upload
await page.setInputFiles('#file',
'path/to/file.pdf');
// Multiple files
await page.setInputFiles('#file',
['file1.pdf', 'file2.jpg']);
```

### ■ Keyboard & Mouse

```
await page.keyboard.press('Escape');
await page.keyboard.type('Hello World');
await page.keyboard.down('Shift');
await page.keyboard.up('Shift');

await page.mouse.click(100, 200);
await page.mouse.dblclick(100, 200);
await page.mouse.move(100, 200);
await page.mouse.wheel(0, 300); // scroll
```

## 5. NAVIGATION & PAGE OPERATIONS

### ■ Navigation

```
await page.goto('https://example.com');
await page.goto('/login'); // uses baseUrl
await page.goBack();
await page.goForward();
await page.reload();

// Wait for navigation
await page.goto(url, {
  waitUntil: 'networkidle' // or 'load',
  'domcontentloaded'
});

await page.url(); // current URL
await page.title(); // page title
```

### ■ Scroll & Viewport

```
// Scroll element into view
await
page.locator('#el').scrollIntoViewIfNeeded();

// Scroll page
await page.evaluate(() =>
window.scrollTo(0, document.body.scrollHeight));

// Scroll by amount
await page.mouse.wheel(0, 500);

// Set viewport
await page.setViewportSize({ width: 1280, height:
720 });

// Emulate device
const { devices } = require('@playwright/test');
use: { ...devices['iPhone 14'] }
```

## 6. ASSERTIONS — expect()

### ■ Page Assertions

```
await expect(page).toHaveTitle('Dashboard');
await expect(page).toHaveTitle(/dash/i); // regex
await expect(page).toHaveURL('https://...');
await expect(page).toHaveURL(/\/dashboard/);
```

### ■ Locator Assertions

```
const loc = page.locator('#el');

await expect(loc).toBeVisible();
await expect(loc).toBeHidden();
await expect(loc).toBeEnabled();
await expect(loc).toBeDisabled();
await expect(loc).toBeChecked();
await expect(loc).toBeFocused();
await expect(loc).toBeEmpty();
await expect(loc).toBeEditable();
await expect(loc).toHaveText('Hello');
await expect(loc).toContainText('ello');
await expect(loc).toHaveValue('input val');
await expect(loc).toHaveAttribute('href',
  '/home');
await expect(loc).toHaveClass('active');
await expect(loc).toHaveCSS('color', 'red');
await expect(loc).toHaveCount(3);
await expect(loc).toHaveId('submit');
```

### ■ Soft Assertions

```
// Soft: collects failures, does not stop test
await expect.soft(page).toHaveTitle('Home');
await expect.soft(loc).toBeVisible();

// Check failures manually
expect(test.info().errors).toHaveLength(0);
```

### ■ Negation & Custom Timeout

```
// Negate any assertion
await expect(loc).not.toBeVisible();
await expect(loc).not.toHaveText('Error');

// Custom timeout for this assertion only
await expect(loc).toBeVisible({ timeout: 5000 });

// Poll until condition (custom assertion)
await expect.poll(async () => {
  return await page.evaluate(() =>
    document.readyState);
}).toBe('complete');
```

### ■ Generic expect

```
expect(value).toBe(5);
expect(value).toEqual({ a: 1 });
expect(arr).toContain('item');
expect(str).toMatch(/regex/);
expect(fn).toThrow();
expect(value).toBeTruthy();
expect(value).toBeFalsy();
expect(num).toBeGreaterThan(3);
```

## 7. WAITS & AUTO-WAITING

■ Playwright auto-waits for elements to be actionable before click/fill/etc. Explicit waits are rarely needed!

| Wait Method                                  | When to Use                                                       |
|----------------------------------------------|-------------------------------------------------------------------|
| Auto-wait (built-in)                         | All actions (click, fill, etc.) auto-wait – no setup needed       |
| <code>await expect(loc).toBeVisible()</code> | Wait for element to appear (preferred over <code>waitFor</code> ) |
| <code>await page.waitForSelector(sel)</code> | Wait for CSS selector to appear in DOM                            |
| <code>await page.waitForURL(url)</code>      | Wait for URL to match string/regex after navigation               |
| <code>await page.waitForLoadState()</code>   | Wait for 'load'   'domcontentloaded'   'networkidle'              |
| <code>await page.waitForResponse(url)</code> | Wait for a specific network response                              |
| <code>await page.waitForRequest(url)</code>  | Wait for a specific network request                               |
| <code>await page.waitForFunction(fn)</code>  | Wait for JS expression to return truthy                           |
| <code>await page.waitForTimeout(ms)</code>   | Hard wait – AVOID, use only as last resort                        |
| <code>await loc.waitFor()</code>             | Wait for locator to be visible/attached/hidden/detached           |

### ■ Wait Examples

```
// Wait for element to appear
await page.locator('#result').waitFor({ state: 'visible' });

// Wait for element to disappear
await page.locator('.spinner').waitFor({ state: 'hidden' });

// Wait for navigation
await Promise.all([
  page.waitForURL('**/dashboard'),
  page.click('#login-btn'),
]);

// Wait for network response
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/user')),
  page.click('#load-btn'),
]);
const data = await response.json();
```



## 8. DIALOGS · FRAMES · TABS · DOWNLOADS

### ■ Dialogs (Alert/Confirm/Prompt)

```
// Accept all dialogs
page.on('dialog', d => d.accept());

// Dismiss dialogs
page.on('dialog', d => d.dismiss());

// Handle with message
page.on('dialog', async d => {
  console.log(d.message());
  await d.accept('typed value'); // for prompt
});
```

### ■ New Tabs / Popup Windows

```
// Capture popup on click
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('#open-tab'),
]);
await newPage.waitForLoadState();
console.log(await newPage.title());

// All open pages
const pages = context.pages();
```

### ■ iFrames

```
// By name or URL
const frame = page.frame('frameName');
const frame = page.frameLocator('#iframe');

// Interact inside frame
await frame.locator('#btn').click();
await frame.fill('#input', 'val');

// Nested frames
const inner = frame.frameLocator('#nested');
```

### ■ File Download

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.click('#download-btn'),
]);
// Save to disk
await download.saveAs('/tmp/file.pdf');
// Get suggested filename
console.log(download.suggestedFilename());
```

## 9. PAGE OBJECT MODEL (POM)

### ■ Page Class

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly username: Locator;
  readonly password: Locator;
  readonly loginBtn: Locator;

  constructor(page: Page) {
    this.page = page;
    this.username = page.locator('#user');
    this.password = page.locator('#pass');
    this.loginBtn = page.getByRole('button',
      { name: 'Login' });
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(user: string, pass: string) {
    await this.username.fill(user);
    await this.password.fill(pass);
    await this.loginBtn.click();
  }
}
```

### ■ Test using POM

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test.describe('Login Tests', () => {
  let loginPage: LoginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.goto();
  });

  test('valid login', async ({ page }) => {
    await loginPage.login('admin', 'pass');
    await expect(page)
      .toHaveTitle('Dashboard');
  });

  test('invalid login', async ({ page }) => {
    await loginPage.login('x', 'wrong');
    await expect(
      page.locator('.error'))
      .toBeVisible();
  });
});
```

## 10. FIXTURES & BROWSER CONTEXT

### ■ Custom Fixtures

```
import { test as base } from '@playwright/test';
import { LoginPage } from './LoginPage';

type MyFixtures = {
  loginPage: LoginPage;
};

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    const lp = new LoginPage(page);
    await lp.goto();
    await use(lp); // provide to test
  },
});

export { expect } from '@playwright/test';

// Use in test:
test('uses fixture', async ({ loginPage }) => {
  await loginPage.login('admin', 'pass');
});
```

### ■ Browser Context

```
// Each test gets a fresh context by default

// Create context manually
const context = await browser.newContext({
  storageState: 'auth.json', // saved auth
  viewport: { width:1280, height:720 },
  locale: 'en-US',
  timezoneId: 'Asia/Kolkata',
  permissions: ['geolocation'],
  geolocation: { lat:12.97, lng:77.59 },
  httpCredentials: { username:'u', password:'p' },
  ignoreHTTPSErrors: true,
});
const page = await context.newPage();
```

### ■ Save & Reuse Auth State

```
// 1. Login once and save state
await page.goto('/login');
await page.fill('#u', 'admin');
await page.fill('#p', 'pass');
await page.click('#btn');
await page.context().storageState({ path:
  'auth.json' });

// 2. Reuse in config
use: { storageState: 'auth.json' }
```

## 11. NETWORK INTERCEPTION & API MOCKING

### ■ Intercept & Mock Response

```
// Mock an API endpoint
await page.route('**/api/users', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([ { id:1, name:'Mock' } ]),
  });
});

// Modify response on the fly
await page.route('**/api/data', async route => {
  const response = await route.fetch();
  const json = await response.json();
  json.extra = 'injected';
  await route.fulfill({ response, json });
});
```

### ■ Capture Network Requests

```
// Listen to all requests
page.on('request', req => {
  console.log(req.method(), req.url());
});

// Listen to all responses
page.on('response', res => {
  console.log(res.status(), res.url());
});

// Wait for specific response
const [resp] = await Promise.all([
  page.waitForResponse('**/api/login'),
  page.click('#login-btn'),
]);
const body = await resp.json();
console.log(body.token);
```

### ■ Abort Requests

```
// Block images & ads
await page.route('**/*. {png,jpg,svg}',
  route => route.abort());

await page.route('**/analytics/**',
  route => route.abort());

// Remove a specific route
await page.unroute('**/api/users');
```

### ■ Direct API Requests (APIRequestContext)

```
const { request } = require('@playwright/test');

test('API test', async ({ request }) => {
  const res = await request.post('/api/login', {
    data: { user: 'admin', pass: 'pass' }
  });
  expect(res.status()).toBe(200);
  const body = await res.json();
  expect(body.token).toBeTruthy();
});
```

## 12. SCREENSHOTS · VIDEO · TRACE VIEWER

### ■ Screenshots

```
// Full page screenshot
await page.screenshot({
  path: 'ss/page.png',
  fullPage: true
});

// Element screenshot
await page.locator('#chart').screenshot({
  path: 'ss/chart.png'
});

// As buffer (for attachments)
const buf = await page.screenshot();
await test.info().attach('screenshot', {
  body: buf, contentType: 'image/png'
});
```

### ■ Video Recording

```
// In playwright.config.ts
use: {
  video: 'on', // always record
  video: 'off', // never record
  video: 'retain-on-failure', // recommended
  video: 'on-first-retry',
}

// Save video after test
await page.video()?.saveAs('test.webm');
```

### ■ Trace Viewer

```
// In config
use: { trace: 'on-first-retry' }

// Manually
await context.tracing.start({
  screenshots: true, snapshots: true
});
// ... run test ...
await context.tracing.stop({
  path: 'trace.zip'
});

// View trace
npx playwright show-trace trace.zip
```

## 13. PARALLEL EXECUTION & CLI COMMANDS

### ■ Parallel Config

```
// playwright.config.ts
export default defineConfig({
  workers: 4, // parallel workers
  fullyParallel: true, // file-level parallelism
  retries: 2,
});

// Force serial for a describe block
test.describe.serial('Sequential', () => {
  test('step 1', ...);
  test('step 2', ...);
});

// Disable parallelism for a single test
test.describe.configure({ mode: 'serial' });
```

### ■ CLI Commands

```
npx playwright test # run all
npx playwright test login.spec.ts # specific file
npx playwright test --grep 'smoke' # by title
npx playwright test --grep-invert 'wip'
npx playwright test --project chromium
npx playwright test --headed # show browser
npx playwright test --debug # debug mode
npx playwright test --ui # UI mode
npx playwright test --workers 4
npx playwright test --retries 2
npx playwright test --reporter html
npx playwright show-report # open HTML report
npx playwright codegen https://app.com # record test
```

## 14. INTERVIEW HOT TOPICS

| Question                                                     | Answer                                                                                                                                                                                 |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Playwright vs Selenium?                                      | Playwright: built-in auto-wait, faster, native multi-browser, network mocking, trace viewer. Selenium: wider language support, mature ecosystem.                                       |
| What is auto-waiting?                                        | Playwright automatically waits for elements to be visible, stable, and actionable before performing any action. No explicit waits needed in most cases.                                |
| page vs context vs browser?                                  | browser = browser instance. context = isolated session (like incognito tab). page = single tab. Each test gets its own fresh context.                                                  |
| How to handle multiple tabs?                                 | <code>context.waitForEvent('page')</code> to capture new tab, then interact via <code>newPage</code> variable.                                                                         |
| Soft assertions?                                             | <code>expect.soft()</code> – does not stop test on failure. All soft assertion failures reported at test end.                                                                          |
| How to reuse auth state?                                     | Login once, save <code>storageState</code> to <code>auth.json</code> , reference in config: <code>use: { storageState: 'auth.json' }</code> .                                          |
| What is a Fixture?                                           | Custom <code>test.extend()</code> setup – reusable, typed setup/teardown per test. Built-ins: <code>page</code> , <code>browser</code> , <code>context</code> , <code>request</code> . |
| How to mock APIs?                                            | <code>page.route('**/api/url', route =&gt; route.fulfill({ status: 200, body: '...' })))</code>                                                                                        |
| What is Trace Viewer?                                        | A timeline replay of a test run (screenshots, actions, network). Enables: <code>use: { trace: 'on-first-retry' }</code> , view with <code>npx playwright show-trace</code> .           |
| Playwright Codegen?                                          | <code>npx playwright codegen</code> – records your browser actions and auto-generates test code.                                                                                       |
| How to run only smoke tests?                                 | <code>npx playwright test --grep '@smoke'</code> – use <code>@smoke</code> in test name or use <code>test.describe</code> tags.                                                        |
| locator vs <code>\$()</code> vs <code>querySelector</code> ? | Always use <code>locator()</code> – it's lazy, auto-retrying, and composable. Avoid <code>\$()</code> and <code>evaluate(querySelector)</code> for test assertions.                    |

## 15. MISC — EVAL · COOKIES · SHADOW DOM · VISUAL

### ■ JavaScript evaluate()

```
// Run JS in browser context
const title = await page.evaluate(() =>
document.title);

// Pass argument to JS
const text = await page.evaluate(
el => el.textContent,
await page.locator('#el').elementHandle());

// evaluateHandle – returns JSHandle
const handle = await page.evaluateHandle(
() => document.body);
```

### ■ Shadow DOM

```
// Playwright pierces shadow DOM by default
// with CSS selectors using >> syntax:
await page.locator('#host >> .inner').click();

// Or use getBy* methods – they pierce shadow DOM
await page.getByRole('button', { name: 'Buy'
}).click();

// Deep pierce with >>> (all shadow roots)
await page.locator('>>> .deep-btn').click();
```

### ■ Cookies & Storage

```
// Get all cookies
const cookies = await context.cookies();

// Set cookies
await context.addCookies([{
name: 'auth', value: 'token',
domain: 'app.com', path: '/',
}]);

// Clear cookies
await context.clearCookies();

// LocalStorage via evaluate
await page.evaluate(() =>
localStorage.setItem('key', 'val'));
```

### ■ Visual Regression (Screenshots)

```
// Compare to baseline screenshot
await expect(page).toHaveScreenshot('home.png');

// Element snapshot
await expect(page.locator('#chart'))
.toHaveScreenshot('chart.png');

// With threshold (% of pixels allowed to differ)
await expect(page).toHaveScreenshot({
maxDiffPixelRatio: 0.05
});

// Update baselines
npx playwright test --update-snapshots
```